

Projet d'Architecture Back-End

1. Vision du projet

Concevoir l'architecture back-end de surveillance et de pilotage d'une plateforme dédiée à un jeu vidéo compétitif en ligne, permettant de suivre, analyser et contrôler l'activité des joueurs, le matchmaking, l'économie virtuelle et le contenu communautaire.

Le projet ne vise pas l'implémentation exhaustive, mais la **définition, justification et structuration** d'un backend **réaliste, scalable et sécurisé**.

2. Objectifs pédagogiques

- Savoir **concevoir une architecture backend complexe**
 - Découper un système en **domaines métier cohérents**
 - Choisir des **patterns architecturaux adaptés**
 - Anticiper la **montée en charge**
 - Penser la **sécurité, la robustesse et l'évolutivité**
 - Justifier ses choix techniques à l'oral et à l'écrit
-

3. Périmètre architectural global

- Type d'architecture (monolithe modulaire, microservices, hybride)
 - Vision des flux entrants/sortants
 - Rôle du backend dans l'écosystème jeu + web
 - Contraintes temps réel vs différé
-

4. Découpage par domaines (Domain-Driven Design)

- Domaine Identité & Accès
 - Domaine Joueurs & Profils
 - Domaine Matchmaking & Compétition
 - Domaine Classements & Progression
 - Domaine Économie Virtuelle
 - Domaine Contenu Communautaire (UGC)
 - Domaine Administration & Modération
-

5. Architecture des services

- Responsabilités de chaque service
 - Frontières entre services/domaines
 - Communications inter-services
 - Gestion des dépendances
-

6. Architecture des API

- Organisation des endpoints
 - Versionning de l'API
 - Stratégies de sécurisation
 - Gestion de la cohérence métier côté serveur
-

7. Gestion des données

- Séparation des modèles métier
 - Choix des technologies de persistance
 - Stratégies de lecture/écriture
 - Gestion des données critiques
-

8. Matchmaking & calculs sensibles

- Positionnement du matchmaking dans l'architecture
 - Traitement synchrone vs asynchrone
 - Isolation des algorithmes critiques
 - Résilience en cas de charge ou d'erreur
-

9. Événements & communication interne

- Usage des événements métier
 - Propagation des changements d'état
 - Découplage des processus
 - Traçabilité des actions
-

10. Sécurité backend

- Stratégie d'authentification
- Autorisation par rôle et par contexte
- Protection des flux sensibles

- Gestion des abus et comportements à risque
-

11. Cohérence et intégrité métier

- Renforcements des règles métier
 - Gestion des transactions
 - Prévention des incohérences
 - Règles serveurs non contournables
-

12. Scalabilité & performances

- Identification des points critiques
 - Stratégies de montée en charge
 - Cache et optimisation
 - Répartition des charges
-

13. Observabilité & exploitation

- Logs fonctionnels et techniques
 - Monitoring des services
 - Suivi des métriques clés
 - Alertes et diagnostic
-

14. Tests d'architecture

- Tests des contrats API
 - Tests d'intégration inter-services
 - Validation des flux critiques
 - Stratégies de non-régression
-

15. Sécurité, fiabilité et résilience

- Gestion des erreurs globales
 - Tolérance aux pannes
 - Reprise sur incident
 - Continuité de service
-

16. Contraintes réalistes & hypothèses

- Hypothèses de trafic
 - Hypothèses d'usage joueur
 - Contraintes techniques simulées
 - Arbitrages coûts / complexité
-

17. Livrables attendus

- Schéma d'architecture global
- Description des domaines et services
- Diagrammes (flux, séquence, composants)

- Justification des choix techniques
 - Présentation orale type comité technique
-

18. Option microservice de monitoring autonome

Dans le cadre de ce projet, il est **vivement recommandé** (et valorisé dans la notation) de réfléchir à une variante où la brique de **surveillance / monitoring** est conçue comme **un microservice indépendant** du reste de la plateforme.

Les étudiants devront :

- **Justifier le choix du microservice de monitoring**
 - Expliquer en quoi il est pertinent de séparer la brique de monitoring du reste du backend.
 - Montrer en quoi cette approche s'intègre bien dans une architecture microservices ou hybride.
- **Garantir l'indépendance vis-à-vis de l'application surveillée**
 - Le microservice doit pouvoir **surveiller n'importe quelle application** (jeu, API backend, service externe, job de traitement, etc.), pas seulement le jeu Nebula.
 - En cas de problème sur le microservice (panne, surcharge, bug, déploiement raté), il **ne doit en aucun cas dégrader le fonctionnement** de l'application surveillée.
 - Les modes de communication choisis (pull/push, collecte de métriques, agents, webhooks, files de messages...) doivent être **non intrusifs** pour l'application monitorée.
- **Définir les responsabilités du microservice**
 - Collecte de métriques (latence, taux d'erreur, volume de requêtes, état des files, etc.).
 - Gestion des seuils et déclenchement d'alertes (ex. temps de réponse trop long, erreurs 5xx, chute d'activité).

- Exposition d'API ou de tableaux de bord pour l'équipe technique / d'exploitation.
- **Penser la résilience et la robustesse du microservice**
 - Stratégie de tolérance aux pannes (redémarrage automatique, réplication, circuit breaker vis-à-vis des applications surveillées).
 - Gestion de la charge sur le microservice sans provoquer de surcharge sur l'application surveillée (backoff, limitation de fréquence, cache).
- **Montrer son intégration dans l'architecture globale**
 - Diagramme de composants / de séquence montrant comment le microservice observe les autres services ou applications.
 - Description des **flux de données** (qu'est-ce qui remonte, à quelle fréquence, via quels protocoles).
 - Justification des choix techniques (stack de monitoring, formats d'événements, protocoles, etc.).

Cette partie sera évaluée sur la capacité à :

- Concevoir un microservice de monitoring **réutilisable** et **générique**.
- Garantir une **isolation forte** entre le système de surveillance et les applications surveillées.
- Argumenter sur les **bénéfices concrets** de cette approche (sécurité, fiabilité, exploitation, évolutivité).

Point clé d'évaluation

- la **clarté de l'architecture**
- la **pertinence des choix**
- la **capacité à anticiper les problèmes**
- la **qualité du raisonnement technique**